Declarations

26 October 2025 10:04

Here we learn how to declare a generic class. It describes constructors, static members, and nested classes, and it fills in some details of how erasure works.

Constructors

In a generic class, type parameters appear in the header that declares the class, but not in the constructor:

```
public class Pair<T, U>
{
   private final T first;
   private final U second;

   public Pair(T first, U second){
      this.first = first;
      this.second = second;
   }

   public T getFirst(){
      return first;
   }

   public U getSecond(){
      return second;
   }
}
```

The type parameters T and U are declared at the beginning of the class, not in the constructor. However, actual type parameters are passed to the constructors whenever it is invoked:

```
Pair<String, Integer> pair1 = new Pair<String, Integer>
    ("one", 1);
```

```
public class TestPair
{
   public static void main(String[] args){
      Pair<String, Integer> p1 = new Pair<>("one", 1);
      Pair<String, Integer> p2 = new Pair<>("two", 2);

      System.out.print("\f p1 = (" + p1.getFirst()+", " + p1.getSecond()+")");
      System.out.print("\n p2 = (" + p2.getFirst()+", " + p2.getSecond()+")");
   }
}
```

```
Type safety: The constructor Pair(Object, Object) belongs to the raw type Pair.
References to generic type Pair<T,U> should be parameterized Java(16777746)

Type safety: The expression of type Pair needs unchecked conversion to conform to
Pair<String,Integer> Java(16777748)

Declarations.Pair.Pair(Object first, Object second)
View Problem (Alt+F8)   Quick Fix... (Ctrl+.)
      Pair<String, Integer> p3 = new Pair(first:"three", second:3);
      System.out.print("\f p1 = (" + p1.getFirst()+", " + p1.getSecond()+")");
      System.out.print("\n p2 = (" + p2.getFirst()+", " + p2.getSecond()+")");
      System.out.print("\n p2 = (" + p3.getFirst()+", " + p3.getSecond()+")");
   }
```

Object Object

This mistake produces a warning, but not an error. It is taken to be legal because Pair is treated as a raw type, a type containing no parametric type information, but which can be converted to the corresponding parameterized type,

generating only an unchecked warning. (Which explains Raw types and how the -Xlint: unchecked flag can help you to spot errors of this kind).

The records can be parameterized in just the same way as regular classes:

```
record Pair<T,U>(T first, U second){}
```

Static Members

Compilation by erasure means that at runtime parameterized types are replaced by the corresponding raw type: for example the interfaces List<Integer>, List<String> and List<List<String>> are all implemented by a single interface, namely List:

```
jshell> import java.util.*;

jshell> List<Integer> ints = Arrays.asList(1,2,3);
ints ==> [1, 2, 3]

jshell> List<String> strings = Arrays.asList("one", "two", "three");
strings ==> [one, two, three]

jshell> ints.getClass() == strings.getClass()
$8 ==> true
```

We see here that the class object associated with a list of integers at runtime. Is same as the one associated with a list of strings.

One consequence is that static members of a generic class are shared across all instantiations of that class, including instantiation at different types. Static members of a class cannot refer to the type parameter of a genre class, and when accessing a static member, the class name should not be parameterized. So let me give you the example:

```
package Declarations;
import java.util.concurrent.atomic.AtomicInteger;
public class Cell<T> {
   private final int id;
   private final T value;
   private final static AtomicInteger count = new AtomicInteger();
   private static int nextId(){
      return count.getAndIncrement();
   }
   public Cell(T value){
      this.value = value;
      id = nextId();
   }
   public T getValue(){
      return value;
   }
   public int getId(){
      return id;
   }
   public static int getCount(){
      return count.get();
   }
}
```

A static field count, is used to allocate a distinct identifier to each cell. For the count, an AtomicInteger is used to ensure that unique identifiers are generated even under concurrent access.

The static getCount() method returns the current count.

```
jshell> Cell<String> a = new Cell<>("One");
a ==> Declarations.Cell@dc24521

jshell> Cell<Integer> b = new Cell<>(2);
b ==> Declarations.Cell@7e0b0338

jshell> a.getId()
$4 ==> 0
```

```
jshell> b.getId();
$5 ==> 1

jshell> Cell.getCount()
$6 ==> 2

jshell> Cell<Double> c  = new Cell<>(3.14);
c ==> Declarations.Cell@3c0f93f1

jshell> c.getId();
$8 ==> 2

jshell> b.getId()
$9 ==> 1

jshell> a.getId()
$10 ==> 0

jshell> Cell.getCount()
$11 ==> 3

jshell>
```

Static members are shared across all instantiation of a class, so the same count is incremented when allocating either a string or an integer cell. Because static members are independent of any type parameters, we are not permitted to follow the class name with the type parameters when accessing a static member:

```
jshell> Cell<Integer>.getCount();
|  Error:
|  illegal start of type
|  Cell<Integer>.getCount();
|                        ^

jshell> Cell<?>.getCount();
|  Error:
|  illegal start of type
|  Cell<?>.getCount();
|                 ^
```

The count is static, so it is a property of a class as a whole, not any particular instance.
For the same reason you can't refer to a type parameter anywhere in the declaration of a static member. Here is the second version of Cell which attempts to use a static variable to keep a list of all values stored in any cell:

```
package Declarations;
import java.util.List;
public class AnotherCell<T> {
   private final T value;
   private static List<T> values = new ArrayList<T>(); illegal
   public AnotherCell(T value){
      this.value = value;
      values.add(value);
   }
   public T getValue(){
      return value;
   }
   public static List<T> getValues(){ //illegal
      return values;
   }
}
```

Since the class may be used with different type parameters at different places. It makes no sense to refer to T in the declaration of the static field values or the static method get values, and these lines are reported as errors at compile time. If we want to list of all values kept in cells, then we need to use a list of objects as in the following variant:

```
package Declarations;
import java.util.ArrayList;
import java.util.List;
public class AnotherCell<T> {
   private final T value;
   // private static List<T> values = new ArrayList<T>(); //illegal
   private static List<Object> values = new ArrayList<Object>(); //OK
   public AnotherCell(T value){
      this.value = value;
      values.add(value);
   }
   public T getValue(){
      return value;
   }
   public static List<Object> getValues(){ //Ok
      return values;
   }
}
```

Output:

```
jshell> import Declarations.*;

jshell> AnotherCell<String> a = new AnotherCell<>("one");
a ==> Declarations.AnotherCell@60c6f5b

jshell> AnotherCell<Integer> a = new AnotherCell<>(2);
a ==> Declarations.AnotherCell@3c0f93f1

jshell> AnotherCell.getValues()
$4 ==> [one, 2]
```

Nested Classes

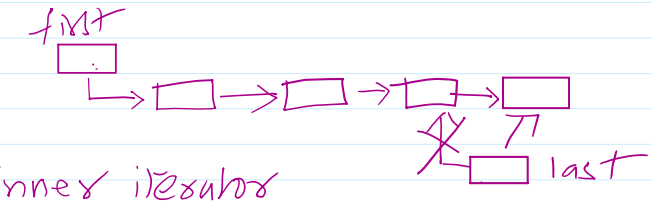
Java permits nesting one class inside another. If the outer class has type parameters and the inner class is a member class that is not static, then type parameters of the outer class are visible within the inner class.

```
package Declarations;
import java.util.AbstractCollection;
import java.util.Collection;
import java.util.Iterator;
import java.util.NoSuchElementException;
public class  LinkedCollection<E> extends AbstractCollection<E> {
   private class Node{
      private E element;
      private Node next = null;
      private Node(E elt){
         element = elt;
      }
   }
   private Node first = new Node(null);
   private Node last = first;
   private int size = 0;
   public LinkedCollection(){}
```

inner class

Shows a class implementing collections as a singly linked list. This class extends java.util.AbstractCollection, so it only needs to define the methods size, add and iterator. It contains an inner class Node for the list nodes and an anonymous inner class implementing Iterator<E> The type parameter E is in the scope within both of these classes.

Iterator<E> The type parameter E is in the scope within both of these classes.




```
  private Node last = first;
  private int size = 0;
  public LinkedCollection(){}
  public LinkedCollection(Collection<? extends E> c){
    addAll(c);
  }

  @Override
  public Iterator<E> iterator() {
    return new Iterator<E>(){
      private Node current = first;
      public boolean hasNext(){
        return current.next != null;
      }
      public E next(){
        if(current.next != null){
          current = current.next;
          return current.element;
        }
        else{
          throw new NoSuchElementException();
        }
      }
      public void remove(){
        throw new UnsupportedOperationException();
      }
    };
  }
  @Override
  public int size() {
    return size;
  }

  public boolean add(E elt){
    last.next = new Node(elt);
    last = last.next;
    size++;
    return true;
  }
}
```

anonymous inner iterator class.

```
jshell> Declarations.LinkedCollection<Integer> li = new Declarations.LinkedCollection<>();
li ==> []

jshell> li.add(10);
$2 ==> true

jshell> li.add(20);
$3 ==> true

jshell> li.add(30);
$4 ==> true

jshell> li.size();
$5 ==> 3

jshell> java.util.Iterator iter =  li.iterator();
iter ==> Declarations.LinkedCollection$1@a67c67e
```

```
jshell> iter.next();
$7 ==> 10

jshell> iter.next();
$8 ==> 20

jshell> iter.next();
$9 ==> 30

jshell> iter.next();
|  Exception java.util.NoSuchElementException
|        at LinkedCollection$1.next (LinkedCollection.java:39)
|        at (#10:1)

jshell>
```

```
package Declarations;
import java.util.AbstractCollection;
import java.util.Collection;
import java.util.Iterator;
import java.util.NoSuchElementException;
public class LinkedStaticCollection<E> extends AbstractCollection<E> {
   private static class Node<T> {
      private T element;
      private Node<T> next = null;
      private Node(T elt) {
         element = elt;
      }
   }
   private Node<E> first = new Node<E>(null);
   private Node<E> last = first;
   private int size = 0;
   public LinkedStaticCollection() {
   }
   public LinkedStaticCollection(Collection<? extends E> c) {
      addAll(c);
   }
   private static class LinkedIterator<T> implements Iterator<T> {
      private Node<T> current;
      public LinkedIterator(Node<T> first){
         current = first;
      }
      public boolean hasNext() {
         return current.next != null;
      }
      public T next() {
         if (current.next != null) {
            current = current.next;
            return current.element;
         } else {
            throw new NoSuchElementException();
         }
      }
      public void remove() {
         throw new UnsupportedOperationException();
```

A similar implementation, but this time the inner Node class is static. And so the type parameter E is not in scope for this class. Instead, the inner class is declared with its own type parameter T. Where the previous version is referred to node, the new version refers to node <E > The anonymous iterator class in the preceding example has also been replaced by a static inner class, again with its own type parameter.

If the node classes has been made public rather than private, you would refer to the node class in the first Example as: LinkedCollection<E>.Node, Whereas you would refer to the Node class in second example as. LinkedStaticCollection.Node<E>.

```
        }
    }
    @Override
    public int size() {
        return size;
    }
    public boolean add(E elt) {
        last.next = new Node<E>(elt);
        last = last.next;
        size++;
        return true;
    }
    public Iterator<E> iterator(){
        return new LinkedIterator<E>(first);
    }
}
```

Output:

```
jshell> Declarations.LinkedStaticCollection<Integer> li1 = new Declarations.LinkedStaticCollection<>();
li1 ==> []

jshell> li1.add(100)
$2 ==> true

jshell> li1.size();
$3 ==> 1

jshell> li1.add(200);
$4 ==> true

jshell> li1.size();
$5 ==> 2

jshell> li1.add(300);
$6 ==> true

jshell> li1.size();
$7 ==> 3

jshell> li1.iterator().hasNext();
$8 ==> true

jshell> java.util. Iterator iter = li1.iterator();
iter ==> Declarations.LinkedStaticCollection$LinkedIterator@4e1d422d

jshell> iter.next();
$12 ==> 100

jshell> iter.next();
$13 ==> 200

jshell> iter.next();
$14 ==> 300

jshell> iter.next();
|  Exception java.util.NoSuchElementException
|        at LinkedStaticCollection$LinkedIterator.next (LinkedStaticCollection.java:45)
|        at (#15:1)
```

Of the two alternative described here, the second is preferable. Member classes are implemented by including a reference to the enclosing instance, since they may in general access component of that instance. Static inner classes are usually both simpler and more efficient.

in Java refers to type erasure, which is a process the Java compiler uses to implement generics.

How Erasure Works

The erasure type is defined as follows: Drop all type parameters from parameterized types and replace any type variable with the erasure of its bounds, or with Object if it has no bounds, or with the erasure of the leftmost bound if it has multiple bounds. Here are some examples :

- The erasure of List<Integer>, List<String>, and List<List<String>> is List.
- The erasure of List<Integer>[] is List[].
- The erasure of List is itself, similarly for any new type.
- The erasure of int is itself, similarly for any primitive type.
- The erasure of Integer is itself, similarly for any type without type parameters.
- The erasure of T in the definition of toList is Object, because T has no bounds.
- The erasure of T in the definition of max is Comparable, because T has bound Comparable<? super T>
- The erasure of T in the final definition of max is Object, because T has bound Object & Comparable<T> and we take the erasure of the leftmost bound.
- The erasure of S and T in the definition of copy are Readable and Appendable because S has bound Readable & Closeable and T has bound Appendable & Closeable.
- The erasure of LinkedCollection<E>.Node or LinkedCollection.Node<E> is LinkedCollection.Node.

In Java, two methods of same class cannot have the same signature, that is the same name and parameter types. Since generics are implemented by erasure, it also follows that two distinct methods cannot have signatures with the same erasure. A class cannot overload two methods whose signature have the same erasure, and a class cannot implement two interfaces that have the same erasure.

For example, here is a class with two convenience methods. One adds together every integer in a list of integers and the other concatenates together every string in a List of String.

```
Erasure of method sum(List<String>) is the same as another method in type
Overloaded Java(16777743)
String Declarations.Overloaded.sum(List<String> strings)
View Problem (Alt+F8)   No quick fixes available
  public static String sum(List<String> strings){
      StringBuffer sum = new StringBuffer();
      for(String s: strings) sum.append(s);
      return sum.toString();
  }
}
```

```
package Declarations;
import java.util.List;
public class Overloaded {
  public static int sum(List<Integer> ints){
    int s = 0;
    for(int i: ints){
      s += i;
    }
    return s;
  }
  public static String sum(List<String> strings){
    StringBuffer sum = new StringBuffer();
```

```
    for(String s: strings) sum.append(s);
    return sum.toString();
  }
}
```

Here the erasure of the declarations of the two methods:

```
int sum(List)
String sum(List)
```

But it is the signature alone, not the return types, that allow the Java compiler to distinguish different method overloads. In this case the erasures of the signatures of both the methods are identical.

So a name class is reported at compile time.

For another example, here is a bad version of Integer class, that tries to make it possible to compare an integer with either an integer or long:

```
class Integer implements Comparable<Integer>, Comparable<Long>{
    // Compile time error cannot implement two interfaces with same erasure.
    private final int value;
    …
    public int comapareTo(Integer i){
    }
    public int comapareTo(Long i){
    }
    …
}
```

If this were supported, it would in general require a complex and confusing definition of bridge methods. The simplest and most understandable option by far to ban this case.